



API Security Testing Lab Report

Target Application: OWASP Juice Shop

Assessment Date: November 20, 2025

Tools Used: Burp Suite, Browser DevTools

Focus Areas: Authentication Bypass, API Vulnerabilities, Privilege Escalation

Introduction

This security assessment of OWASP Juice Shop identified critical API vulnerabilities through methodical testing aligned with the OWASP API Security Top 10. The assessment uncovered severe weaknesses in authentication, authorization, and business logic that could lead to full system compromise.

1. Authentication Bypass & SQL Injection

This section documents the critical authentication vulnerabilities discovered in the Juice Shop login mechanism, focusing on SQL injection attacks that completely bypass authentication controls and allow unauthorized access to user accounts, including administrative privileges.

1.1 SQL Injection in Login Mechanism

The login endpoint demonstrated fundamental SQL injection vulnerabilities allowing attackers to manipulate database queries through unsanitized user input in the email field.

Vulnerability: SQL Injection via login endpoint

- **Payload:** ' or '1'='1'--
- **Technique:** Boolean-based blind SQL injection
- **Result:** Successful authentication bypass
- **Impact:** Unauthorized access to user accounts



Figure 1: SQL injection payload in email field: ' or '1'='1'--

The image shows a login form titled "Login" on a dark background. The form has two input fields: "Email*" and "Password*". The "Email*" field contains the SQL injection payload: `' or '1'='1'--`. The "Password*" field contains four dots, indicating a password is entered, and has a toggle icon (an eye) to the right. Below the password field is a link that says "Forgot your password?". There are two buttons: a blue "Log in" button and a green "Log in with Google" button. Below the buttons is a checkbox labeled "Remember me" which is checked. At the bottom of the form is a link that says "Not yet a customer?".

1.2 Admin Account Compromise

Through SQL comment manipulation, complete administrative access was achieved without valid credentials, demonstrating severe authentication mechanism failures.

Admin Access Method:

- **Email:** admin@juice-sh.op'--
- **Password:** 12345
- **Technique:** SQL comment operator (--) to truncate password verification



- **Outcome:** Full administrative access achieved

Figure 2: Admin account access using SQL comment bypass

The screenshot shows the CYART login interface. The 'Email*' field contains the text 'admin@juice-sh.op'--', which is an SQL comment bypass. The 'Password*' field contains '12345'. Below the password field is a link for 'Forgot your password?'. There is a 'Log in' button with a key icon, a checked 'Remember me' checkbox, and a 'Log in with Google' button. At the bottom, there is a link for 'Not yet a customer?'.

Login

Email*
admin@juice-sh.op'--

Password*
12345

[Forgot your password?](#)

 Log in

☒ Remember me

or

 Log in with Google

[Not yet a customer?](#)



2. Privilege Escalation & Admin Panel Access

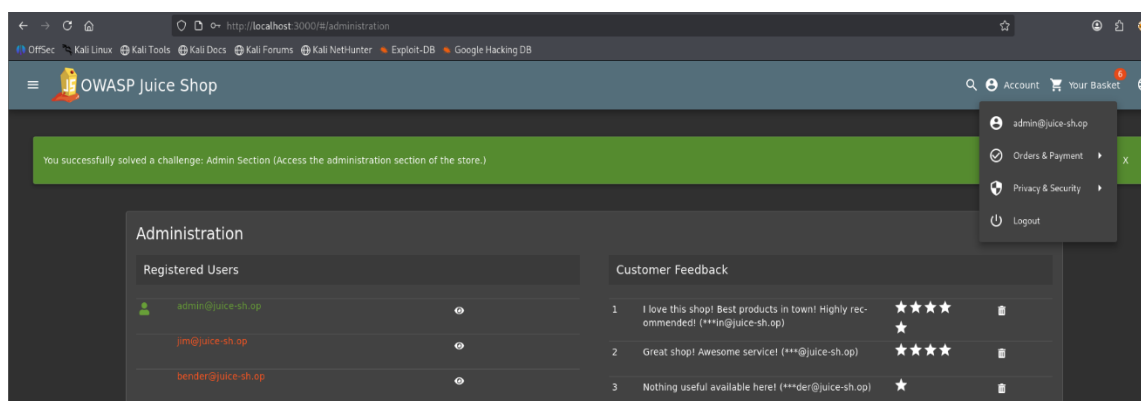
2.1 Administrative Interface Access

Analysis: Successful privilege escalation provided complete access to the administration panel, exposing user management functions and system configuration capabilities.

Administrative Capabilities Gained:

- View all registered users
- Access customer feedback management
- Monitor application statistics
- Perform administrative functions

Figure 3: Successful administrative section access



2.2 User Account Verification

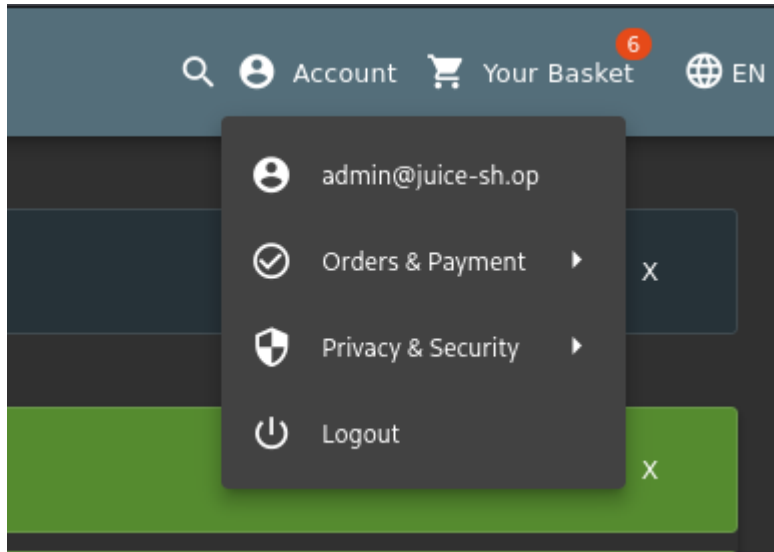
Analysis: Administrative access verification confirmed the ability to manage user accounts, orders, and application settings, highlighting critical authorization failures.

Verified Access Levels:

- Admin account dashboard
- Order management capabilities
- User administration functions
- System configuration access



Figure 4: Confirmed admin account access with full privileges



3. API Manipulation & Business Logic Vulnerabilities

This section examines API endpoint vulnerabilities and business logic flaws that allowed manipulation of application functionality, including user impersonation and unauthorized data modification.

3.1 Feedback Forgery Attack

Analysis: Business logic vulnerabilities enabled posting feedback under other users' identities and bypassing rating validation mechanisms, demonstrating inadequate authorization checks.

Vulnerabilities Exploited:

1. **Feedback Forgery:** Post feedback in another user's name
2. **Zero-Star Rating:** Bypass rating validation
3. **CAPTCHA Bypass:** Client-side validation circumvention

You successfully solved a challenge: Forged Feedback (Post some feedback in another user's name.)

You successfully solved a challenge: Zero Stars (Give a devastating zero-star feedback to the store.)

Customer Feedback

Author

***@juice-sh.op

Comment*

1 Max. 160 characters

0/160

Rating

CAPTCHA: What is 8*6-7 ?

Result*

Figure 6: Raw HTTP request showing manipulated feedback submission

```
Request
Pretty Raw Hex
9 Content-Length: 94
10 Origin: http://192.168.150.128:3000
11 Connection: keep-alive
12 Referer: http://192.168.150.128:3000/
13 Cookie: language=en; welcomebanner_status=dismiss; continueCode=pEMlPweJbS0BQZxrXyAycpfIhMyu3jtWatp3uybsJbdj2wo47LBYV9qaRkYn; cookieconsent_status=dismiss; token=eYJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eYJzdGF0dXMhOiJzdWNjZXNzIiwiaWF0IjE6MTUwMDE2MDAwMCJ9.CmShbWUuO1IiLCJlbnRpbnBiCiEppbUBqdWljZS1zaC5vcCisInBhc3Ns3kJiOi0GE2ZWNVZlCkpwZmxleGVubGUzbGl6IiwiaXNhc3RpdDpkblkiOiwiOiJwcmVudGZlcnZlIiwidWlidCIjoixXNmZl3BJYexpYy9pbWFnZXNhdXBsb2Fkcy9kZWZhdxoLNnZyIsInRvdHBTZWnyZXQiOiJlLCJpcOfjdGdlG2ZSI6dHwogAiWiwidXBKYYXRlREF0ZjoijAyNS0xMCOzMCAwNzowNyoiNy40NzYgKgAwOjAiWiwidGVZsZRlREF0ZjpudWxsflSwiaWF0IjoxxNZsODAMDESEf0.E-Rogs0Qiu2FCklpq2vVAuuUzjYCBrUqrEph290-Qvo9-gW8ORlOB2rRTMR_lVLBo095VDqtIJGkCzhFKAJzIshpaBK3gYwNc7uvnuOV4E7Gc
14 Priority: u=0
16 {
  "UserId":3,
  "captchaId":4,
  "captcha":"41",
  "comment":"'not good enough (***@juice-sh.op)",
  "rating":0
}
```

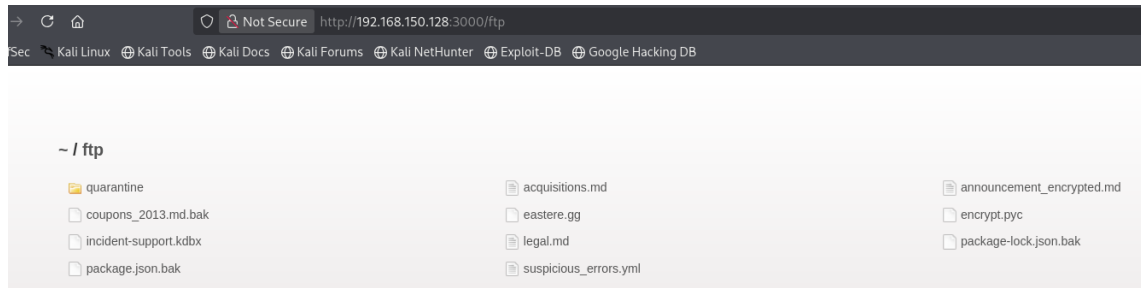
- **Endpoint:** Feedback submission API
- **Vulnerability:** UserID manipulation (BOLA)
- **Parameter Tampering:** UserId changed to 3 for impersonation
- **Token Usage:** JWT token in authorization header



4. File System Discovery & Information Disclosure

4.1 FTP Directory Enumeration

Figure 7: FTP directory listing revealing sensitive files



Critical Files Discovered:

- **incident-support.kdbx** - KeePass database file
- **coupons_2013.md.bak** - Backup containing promotion codes
- **announcement_encrypted.md** - Encrypted announcements
- **encrypt.pyc** - Encryption utility (compiled Python)

5. API Security Testing Results

5.1 Vulnerability Log

Test ID	Vulnerability	Severity	Target Endpoint
001	SQL Injection	Critical	/rest/user/login
002	Broken Object Level Authorization	High	/api/Feedbacks
003	Authentication Bypass	Critical	/rest/user/login
004	Information Disclosure	Medium	/ftp



Completed Assessment Activities:

- ☒ Enumerate API endpoints via proxy interception
- ☒ Test for BOLA using token manipulation
- ☒ SQL injection testing on authentication endpoints
- ☒ Business logic vulnerability assessment
- ☒ File upload/download functionality testing

API Test Summary

The OWASP Juice Shop API assessment revealed critical security flaws including SQL injection allowing complete authentication bypass, broken object level authorization enabling user impersonation, and insecure direct object references exposing sensitive files. The application demonstrated multiple OWASP API Top 10 violations, particularly in authentication and authorization mechanisms. Immediate remediation of SQL injection and authorization flaws is required to prevent complete system compromise.